Using the Dataverse token reader and Power Query workbook

Step-by-step guide • November 2, 2026

This guide covers two related pieces: testing the uncategorized report with a CSV in Excel, and adding your existing Dataverse token source to **BWC – Refresh Orchestrator** in Power Automate. It is an implementation guide, not confirmation that a tenant flow has been deployed.

1. Know what each file does

File	Open it in	Purpose
`Power Query Setup Template.xlsx`	Excel desktop with Power Query	Clean input workbook for one-time installation of the supplied Power Query source; then import CSV, report, and validation queries.
`reports.evidenceCreated_mock.csv`	Read by the Excel queries	The 1,000-record synthetic source.
`data/test-inputs/`	Read by the validation queries	Seven isolated inputs for padding, duplicate IDs/serials, missing ID/header, and invalid date/number tests.
`dataverse-token-settings.template.json`	A text editor	Your Dataverse settings and editable assumptions. This is configuration documentation, not a file to upload to Dataverse.
`dataverse-token-flow.template.json`	A text editor	Action names, settings, and expressions to copy into a flow. It is not an importable Power Automate flow.
`Dataverse Token Reader - Template.md`	A Markdown/text viewer	The token-reader setup reference.
`automation/report-template.xlsx` in RScriptDashboard	SharePoint / Excel Online	The separate cloud-processing workbook for `ProcessEvidence` and incremental state.

The Power Query workbook and the repository's cloud-processing workbook serve different roles. After query installation, the local workbook has report-output tabs with the original data headers. The separate expected-report file supplies the original formatting reference. The cloud workbook has named tables such as `Uncategorized`, `Snapshot`, `DataQuality`, and `EvidenceState`. Use the correct workbook for each set of instructions.

How they fit together

```
LOCAL REPORT TEST
Mock CSV -> Excel Power Query workbook -> report tabs + validation

SCHEDULED CLOUD REPORT
Existing token publisher -> Dataverse token cache
                                    |
                                    v
                     Power Automate reads the token
                                    |
                                    v
                         Axon reporting API
                                    |
                                    v
               Excel Online ProcessEvidence script
                                    |
                                    v
              SharePoint completed report -> Power BI
```

The token is used by the acquisition flow. The CSV test workbook does not need a live token. Power BI reads the published report data.

2. Extract and keep the local files together

1. Extract the supplied workbook/data package into a normal folder on your computer. Do not open the workbook directly inside a ZIP.

2. Keep the package structure:

```
Power Query Workbook/
    Power Query Setup Template.xlsx
    Expected Uncategorized Report.xlsx
    m/
        EvidenceModel.m
        Report_*.m
        Checks_*.m
    data/
        reports.evidenceCreated_mock.csv
        test-inputs/
            padded_valid_input.csv
            duplicate_evidence_id.csv
            duplicate_evidence_serial.csv
            invalid_timestamp.csv
            invalid_numeric.csv
            missing_evidence_id.csv
            missing_required_header.csv
```

3. The combined walkthrough package includes a **Dataverse Token Reader** folder with the same three reference files as `Dataverse Token Reader Template.zip`. Keep them beside the designer while creating the actions manually.

4. Preserve an unchanged copy of the mock source so you can restore it after deliberate tests.

3. Configure the local Power Query setup workbook

1. Open `Power Query Setup Template.xlsx` in **Excel desktop**.

2. Open the **Settings** worksheet.

3. Edit the yellow **Value** cell beside `DataFolder`. Enter the full path to the extracted **data** folder. Examples:

```
Mac:     /Users/yourname/Documents/BWC Test/Power Query Workbook/data
Windows: C:\Users\yourname\Documents\BWC Test\Power Query Workbook\data
```

4. Keep the other baseline settings:

Parameter	Baseline value
`SourceFile`	`reports.evidenceCreated_mock.csv`
`AsOfDate`	`2026-04-27`
`RunFixtureChecks`	`TRUE`
`ExpectedSourceRows`	`1000`
`ExpectedUncategorizedRows`	`757`

5. Keep parameter names and worksheet table names unchanged. They are query dependencies.

6. Save your working copy.

`AsOfDate` is deliberately frozen. Changing it changes the expected ages; it does not update the independent expected results stored in the workbook.

One-time query installation

The setup workbook deliberately contains no installed queries or connections. The earlier experimental embedded-query workbook did not pass native Excel acceptance, so it is not included as a working deliverable. Use this clean setup file plus the supplied M source. The expected report is a static reference, not a completed refresh.

The following load-menu names are for **Excel desktop on Windows**. On Mac, Get Data/Blank Query menus can differ; the local Mac authoring attempt requested a gateway and did not complete a CSV refresh. If that happens, use a supported Windows Excel installation for this local template or continue with the separate cloud-processing route. A gateway is not a required setup step for the cloud architecture described here.

1. Open the **Query Catalog** worksheet. It lists 21 queries: one model query and 20 output queries.

2. Open `m/EvidenceModel.m` in a text editor and copy its entire contents, including `let` through the final `in` expression.

3. In Excel choose **Data → Get Data → From Other Sources → Blank Query**.

4. In Power Query Editor select **Home → Advanced Editor**, replace the starter text, and paste the copied code.

5. Confirm the editor and name the query **EvidenceModel** in Query Settings. Use this exact name.

6. Select **Close & Load → Close & Load To… → Only Create Connection**. The model returns a record of tables; it is not a worksheet table itself.

7. Create another Blank Query. In Advanced Editor paste the full contents of `m/Report_DaysPastDue.m`:

```
EvidenceModel[Tables][DaysPastDue]
```

8. Name this query **Report_DaysPastDue**. Choose **Close & Load To… → Table → New worksheet**. Rename the new worksheet **Days Past Due**.

9. Create **Checks_Validation** from `m/Checks_Validation.m`, which contains `EvidenceModel[Validation]`. Load it as a new worksheet table, then rename that worksheet **Scenario Validation**.

10. Repeat for every remaining output in Query Catalog: the M file name without `.m` is the query name; **Worksheet / destination** is the worksheet name after loading. Load each output as a table on a new worksheet. The catalog includes the original component/AOR/owner reports, Source Audit, Rejected Rows, Mapping Exceptions, and Test Input Results.

11. Save your working workbook. You can then use **Data → Refresh All** for subsequent imports; do not recreate the queries on every run.

Keep **Settings**, **Org Crosswalk**, and the hidden **Expected Report** input tables intact. `tblSettings`, `tblOrgMap`, and `tblExpectedReport` are dependencies of EvidenceModel. The M files are authoring backups after being pasted into the workbook; they are not executed as external scripts during a normal refresh.

Once installed, the queries—not the static reference workbook—produce the report worksheets. Native query refresh has not passed acceptance in this session; use the checks below to establish that result in your supported Excel installation.

4. Refresh and inspect the local report

1. Select **Data → Refresh All**.

2. If Excel requests access to the package's local source files, select the requested file/folder in your extracted package and grant the normal file access needed for the import.

3. Wait until the refresh finishes. Opening a workbook with saved results is not evidence that a refresh completed.

4. Open **Source Audit**. Verify the source path points to your copy and inspect its refresh timestamp, schema status, populated count, rejected count, and uncategorized count.

5. Open **Scenario Validation**. Look at **Status** and **Refreshed UTC**. `NOT REFRESHED` is a pending cache, `FAIL` needs investigation, and `SKIPPED` means the check did not run.

6. Open **Days Past Due** for the main detail output. Query loading creates the data table; use Expected Uncategorized Report.xlsx as the formatting reference.

7. Inspect the component, AOR, age-band, and owner summaries. They are views of the same detail population; do not append all the tabs into one fact table.

8. Open **Mapping Exceptions** to see blank, unmapped, or conflicting groups. Valid records remain in the report when their organization fields cannot be assigned.

9. Open **Rejected Rows** for malformed or later duplicate records. The clean primary fixture should have no rejected records.

Expected local baseline

Measure	Expected
Populated CSV records	1,000
Rejected primary-source records	0
Categorized records excluded	243
Uncategorized records	757
Age greater than 30	302
Age 15–30 inclusive	205
Age less than 15	250

The local fixture uses **critical >30**, while the linked cloud project uses **critical 30+ / moderate 15–29**. These are explicit, different test configurations. Do not silently compare their day-30 totals. The local fixture's report dates and

age calculations use UTC upload calendar dates.

5. Run the isolated CSV tests

With `RunFixtureChecks=TRUE`, the validation queries read the seven `test-inputs` files separately. Their bad records are not appended to the main report.

1. Open **Test Input Results** after a completed refresh.

2. Check the result for each input:

Input	Expected result
`padded_valid_input.csv`	1,000 accepted, 40 empty records ignored, 0 rejected.
`duplicate_evidence_id.csv`	1 accepted; second record rejected for duplicate Evidence ID.
`duplicate_evidence_serial.csv`	1 accepted; second record rejected for duplicate evidence serial.
`invalid_timestamp.csv`	1 accepted; second record rejected for invalid upload timestamp.
`invalid_numeric.csv`	1 accepted; second record rejected for invalid size.
`missing_evidence_id.csv`	1 accepted; second record rejected for missing Evidence ID.
`missing_required_header.csv`	Entire file rejected before row processing; `Categories` is absent.

3. For a deliberate main-source test, change `SourceFile` to a relative path such as `test-inputs/duplicate_evidence_id.csv`, save, and refresh.

4. Expect the fixed 1,000/757 baseline checks to fail for a two-record input. That is not an unexpected result. Inspect Source Audit and Rejected Rows instead.

5. Restore `SourceFile=reports.evidenceCreated_mock.csv` and refresh again before accepting the baseline.

6. For a different real source, use an isolated working copy and set `RunFixtureChecks=FALSE`. Fixture-specific checks become `SKIPPED`; import-quality and reconciliation checks still apply. The 70-column schema contract remains required.

Do not use a missing-header file as a normal report source. Its report-query refresh should fail visibly. Excel can retain prior saved worksheet results after a failed refresh, so check the error and Source Audit rather than reading the old table as a new report.

6. Prepare the Power Automate connections

This begins the cloud part of the setup. Use the Microsoft 365 / government-cloud portals approved for your environment.

1. Open Power Automate and select the intended environment.

2. Locate or create the existing **Microsoft Dataverse** connection for `https://icewbc-dev.crm9.dynamics.com`.

3. The connection identity needs read access to the published token row and its token/expiry columns.

4. Identify the **SharePoint** connection the report flow will use. Give that account **Can edit** access to the intended `BWC Reporting` folder if it does not already have it. This account is the “flow identity” referred to in the setup instructions;

it may differ from the person who owns the flow.

5. For the cloud route, obtain automation/report-template.xlsx from your cloned repository. Upload this file into SharePoint `BWC Reporting/templates/`.

6. Open that uploaded workbook in **Excel for the web** → **Automate** → **New Script**. Replace the starter with ProcessEvidence.office.ts and save it as **ProcessEvidence**. Create another script from ReadRefreshState.office.ts, saved as **ReadRefreshState**. These are script contents to paste, not Excel files to upload.

7. Confirm the **Excel Online (Business)** connection identity can select both scripts and access the working workbook. Script ownership/sharing and workbook sharing are separate.

8. Record your SharePoint site, document library, reporting folder, and Power BI workspace. Use placeholders until these are chosen; do not substitute a sharing link for a SharePoint site URL.

Recommended SharePoint structure for the repository's cloud process:

```
BWC Reporting/
 templates/report-template.xlsx
 raw/
 latest/
```

Upload **the repository's** `automation/report-template.xlsx` into `templates`. The local mock Power Query workbook is not the Office Script processing template.

7. Create the Refresh Orchestrator shell

1. In Power Automate select **Create** → **Scheduled cloud flow**.

2. Name it **BWC – Refresh Orchestrator**.

3. Choose your intended daily or weekly schedule. The repository starts with **6:00 AM, Eastern Standard Time**; this named zone handles Eastern daylight-saving transitions.

4. In **Recurrence** → **Settings**, enable concurrency control and set the degree to **1** so this flow does not publish overlapping runs.

5. Keep the flow turned off between setup/pilot tests. Use the designer's manual test option while configuring it.

6. Rename actions as you add them. The expressions below depend on exact action names with underscores.

7. Add **Data Operation** → **Compose**, named `SnapshotDate`. In its Expression tab enter:

```
formatDateTime(convertTimeZone(utcNow(), 'UTC', 'Eastern Standard Time'), 'yyyy-MM-dd')
```

8. Add another **Compose**, named `RunId`:

```
workflow()['run']['name']
```

Before adding page loops or Conditions, add **Variables** → **Initialize variable** actions at the top level for:

Variable	Type	Initial value
`RunFailed`	Boolean	`false`
`FailureReason`	String	Empty
`PageAccepted`	Boolean	`false`
`TokenRereadForPage`	Boolean	`false`

Variable	Type	Initial value
`FinishedCreated`	Boolean	`false`
`FinishedUpdated`	Boolean	`false`

These extend the repository's initialization block. Reset PageAccepted and TokenRereadForPage at the start of each page. Keep RunFailed false until an error occurs.

Failure pattern inside a page loop: use Set variable to set RunFailed=true and FailureReason to a nonsecret code. Do not put a Terminate action inside a loop. The page retry loop exits on `or(variables('PageAccepted'), variables('RunFailed'))`; each outer report loop exits on `or(variables('FinishedCreated'), variables('RunFailed'))` or the corresponding FinishedUpdated expression. Acquisition and publication continue only when the failure flag is false and the corresponding completion flags are true. Add the actual Failed Terminate actions outside the report loops, as described in Step 10.

Use **dynamic content** or **Expression** tokens for values supplied by prior actions. A literal string that looks like an expression is not the same as an evaluated expression. In the designer's Expression tab, omit the leading @ shown in code-view JSON references.

8. Add the Dataverse token reader

Keep `dataverse-token-flow.template.json` open beside the designer as a reference. Do not try to import that JSON as a flow.

Where the reader runs

For this walkthrough, put the entire token-read-and-guard block **at the start of each report-page iteration**, before its HTTP GET. This keeps each subsequent page on the current published token without storing tokens in ordinary flow variables. It adds one small Dataverse read per page; include those calls in your production-volume check.

Build the Created-page version with the names shown below. In the Updated-page loop, copy the block with distinct action names: use `List_Updated_Page_Axon_Token` for its regular read and `ReRead_Updated_Page_Axon_Token` for a 401 reread, and update each copied expression to reference its corresponding action. Action names must be unique within the flow.

8A. Add List rows

1. At the start of the Created-page loop, after that page's initialization/reset actions, select **Add an action**. The repository's acquisition steps define the loop; use this block before its page GET.

2. Search **Microsoft Dataverse**, then select **List rows**.

3. Rename the action `List_Published_Axon_Token`.

4. Use the Dataverse connection for the supplied environment.

5. Select the table corresponding to logical name `bwc_axonauthcache` / entity set `bwc_axonauthcaches`.

6. Open its advanced parameters and enter:

Setting	Value
Select columns	`AccessToken,ExpiresAtUtc`
Row count	2

Setting	Value
Filter rows	Empty, under your confirmed single-current-row contract

7. In the action's Settings, enable the supported **Secure Inputs** and **Secure Outputs** controls.

`AccessToken` and `ExpiresAtUtc` are the names you supplied. If the connector requires different logical property names, replace them consistently in Select columns and every expression. The template does not guess a publisher prefix.

8B. Require one row

1. Add **Control** → **Condition**, named `Require_Exactly_One_Token_Row`.

2. Use this Expression as the left value, compare **is equal to**, and enter Boolean `true` as an Expression on the right:

```
equals(length(body('List_Published_Axon_Token')?['value']), 1)
```

3. In **If no**, add **Set variable** actions: `RunFailed=true` and `FailureReason=TOKEN_ROW_COUNT_INVALID`. The loop will exit and the outside-loop failure gate will terminate the run.

4. Place the remaining token checks and that page's acquisition steps inside **If yes**. A Condition action can be successful even when its No branch ran, so do not treat the Condition's success status as proof that a token was valid.

Requesting two rows lets this guard detect an unexpected second row. `$top=1` would hide the ambiguity. The `$top=10` URL supplied for describing the table is not a current-token selection rule.

8C. Require a token value

Inside the one-row **If yes** branch:

1. Add a Condition named `Require_Nonempty_Token`.

2. Compare this Expression to Boolean `true`:

```
not(empty(trim(coalesce(first(body('List_Published_Axon_Token')?['value'])?['AccessToken'], ''))))
```

3. In **If no**, set `RunFailed=true` and `FailureReason=TOKEN_EMPTY`; do not place Terminate inside the loop.

4. Continue inside its **If yes** branch. This order matters: do not use `first(...)` before the row-count guard succeeds.

8D. Check expiry

1. Add a Condition that compares this Expression to Boolean `true`:

```
not(empty(first(body('List_Published_Axon_Token')?['value'])?['ExpiresAtUtc']))
```

2. In **If no**, set `RunFailed=true` and `FailureReason=TOKEN_EXPIRY_MISSING`.

3. In its **If yes** branch, add a **Scope** named `Validate_Token_Expiry`.

4. Inside that Scope, add a Condition comparing this Expression to Boolean `true`:

```
greater(
  ticks(first(body('List_Published_Axon_Token')?['value'])?['ExpiresAtUtc']),
  ticks(addSeconds(utcNow(), 120))
)
```

5. In the freshness Condition's **If no** branch, set `RunFailed=true` and `FailureReason=TOKEN_NOT_FRESH`.

6. Add two parallel paths after `Validate_Token_Expiry`. Configure the **acquisition success path** to run after that Scope **is successful only**, then add an `Acquire_Page_If_Valid` Condition requiring `equals(variables('RunFailed'), false)` before its GET. A freshness No branch can leave the Scope successful while RunFailed is true. Configure a **failure-handler Scope** after **has failed** or **has timed out**; it sets `RunFailed=true` and `FailureReason=TOKEN_EXPIRY_INVALID`. This catches timestamp parse errors. Do not output the row's contents or put Terminate inside this loop.

7. Do not make acquisition depend on the failure-handler Scope: that handler will be skipped during a good run. The 120-second margin is editable. The existing token-publisher process owns renewal.

The expiry is assumed to be an ISO 8601 timestamp with UTC or an explicit offset. There is no separate client-secret field or token-request POST in this reader.

9. Put the token into the Axon report request

1. Use your existing HTTP report GET action, or add the GET action while completing the repository's acquisition steps.

2. Add an **Authorization** header.

3. In its value's Expression tab enter:

```
concat('Bearer ', trim(first(body('List_Published_Axon_Token')?['value'])?['AccessToken']))
```

4. This assumes the stored AccessToken is a raw token. If your publisher already stores the full `Bearer ...` header value, use that value directly instead of adding a second prefix.

5. Enable supported secure input/output settings on token-bearing HTTP actions and any other action that exposes the token.

6. Use this read action's current-iteration token for that page request and ordinary transient retries. On the first report-request 401 for the page, re-read the published row and repeat the guards. Retry the same page once only with a usable replacement. The next page starts by executing the regular reader again, so it picks up the replacement without retaining the expired initial value. A missing replacement, second 401, or 403 sets RunFailed=true and an appropriate nonsecret FailureReason; the outside-loop gate fails the run.

Wire the 401 handling in the designer

For each report-page request, use the repository's request/response Scopes so a failed HTTP action can be inspected:

1. Put the original HTTP action in a request Scope. If you use the repository's manual retry controller, set the HTTP action's automatic retry policy to **None** so two retry controllers do not compete.

2. Add a response-handler Scope configured to run after that request Scope **is successful**, **has failed**, or **has timed out**. Read the HTTP action's actual `statusCode`. A failed action is not an empty report page.

3. For the first 401 on that page, use a separate Dataverse List rows action named `ReRead_Published_Axon_Token` and repeat the one-row, nonempty-token, and expiry guards. Require a usable replacement; do not advance the page offset.

4. Use a separate retry HTTP action for the same URL/page. Its Authorization expression must reference the **new** read action, not the first read:

```
concat('Bearer ', trim(first(body('ReRead_Published_Axon_Token')?['value'])?['AccessToken']))
```

5. Handle the retry action's success/failure through its own response Scope. Accept and append a valid page only once. A second 401 sets RunFailed=true and FailureReason=AXON_401_AFTER_REREAD; it must not route back into another token reread. Terminate only at the outside-loop gate.

6. Keep the repository's bounded 429/5xx retry policy for other retryable failures and its failure behavior for 403/configuration errors. Reset the per-page one-time token-reread flag before the next page.

Do not add a diagnostic Compose containing the token, load it into Excel/Power BI, or export it with the report.

10. Complete acquisition, processing, and publication

The token reader completes authentication setup; it does not collect or publish evidence by itself. The existing repository's implementation guide gives the full Created/Updated acquisition and publication steps. Use **its Steps 7–14** with the Dataverse reader above replacing **its direct token-request instructions**.

Complete these blocks in this order:

1. **Choose a new working copy and refresh mode.** Initial/full runs copy `templates/report-template.xlsx`; incremental runs copy the last successful `latest/Uncategorized Report.xlsx` and read its state using `ReadRefreshState`. Only a confirmed missing latest file selects an initial run; permission/service errors fail.

2. **Freeze acquisition windows.** The approved recording-start scope begins `2024-03-11`. Created requests filter recording start, not upload age. Incremental runs use the last successful state and overlap. A recent Created-only pull will miss later category/status changes on older evidence.

3. **Collect EvidenceCreated pages.** Use the documented reporting endpoint, page size at most 50, pageOffset starting at 0. After each accepted nonempty page increment pageOffset by **1**, not 50. Continue through short nonempty pages; stop only after a successful empty `data` array and retain that terminal page.

4. **Collect EvidenceUpdated pages.** Use the Updated window/schema. Updated categories have a different shape from Created categories. Keep the request window fixed during each loop. Retry limits and loop limits are failures, not successful completion.

5. **Add failure gates outside the loops.** After the Created loop, continue only if RunFailed=false and FinishedCreated=true. Otherwise use Control → Terminate with status Failed, outside the loop. If the loop/request block failed or timed out, its outside handler must still run; configure run-after for succeeded, failed, and timed out and set/check failure state. Apply the same gate after the Updated loop, requiring RunFailed=false and FinishedUpdated=true. A loop limit without a validated empty terminal page is a failure, not completion. Then **archive the complete refresh envelope** in the run's `raw/<SnapshotDate>/<RunId>/` folder. Keep failed/partial acquisition out of `latest`.

6. **Run ProcessEvidence** on the same working workbook used to read state. For API backfill/incremental bundles, use `sourceFormat=axon-refresh-bundle`, `csvText=string(outputs('Compose_Completed_Refresh_Bundle'))`, SnapshotDate/RunId outputs, and `sampleData=false` for production. Despite its parameter name, `csvText` contains JSON in this mode.

7. **Check the script result.** Name the Run script action `Process_Evidence`. For a production run, compare this Expression to Boolean `true` before publication:

```
and(
  equals(body('Process_Evidence')?['result']?['run_id'], outputs('RunId')),
  equals(body('Process_Evidence')?['result']?['snapshot_date'], outputs('SnapshotDate')),
  equals(body('Process_Evidence')?['result']?['sample_data'], false),
  equals(body('Process_Evidence')?['result']?['rejected_rows'], 0)
)
```

If false, terminate Failed and inspect the working workbook's Data Quality sheet. Apply this strict gate to production. The separate smoke fixture deliberately has sample_data=true and three rejected rows; verify it with the smoke-test expectations instead of this production gate.

8. **Publish the completed workbook bytes.** Get file content from the completed working copy. Create `latest/Uncategorized Report.xlsx` only for a confirmed first run; update the existing file content on

subsequent runs. Do not delete/recreate the latest file or advance a separate watermark on failure.

9. **Refresh Power BI after publication.** Configure the chosen workspace's supported refresh action and confirm the report's Run ID and snapshot date match the newly published workbook.

For a cloud smoke test before live API acquisition, follow the repository's implementation guide Step 5 using `examples/axon-pages.sample.json`: expected **8 uncategorized, 4 critical, 2 moderate, 2 normal, 3 intentional rejections**. Use a smoke-test folder outside `latest`. Those are not the 757-record local workbook totals.

11. Do not substitute Excel “Refresh All” for cloud processing

For local CSV testing, manually run Excel desktop **Data** → **Refresh All**.

For the recurring desktop-free process, pass the acquired records into **ProcessEvidence** through Excel Online. Microsoft documents that Office Scripts run by Power Automate cannot refresh most Excel data connections; `refreshAllDataConnections` only refreshes Power BI sources. A successful script action calling Refresh All would not prove that this CSV Power Query refreshed. Microsoft: refresh limitations

The report can still run without your desktop through the repository's Power Automate → Excel Online processing → SharePoint → Power BI Service architecture. Do not upload the local CSV workbook and assume it will become that cloud process automatically.

12. Troubleshooting and completion checks

Symptom	What to check
JSON cannot be imported as a flow	The token JSON is an action reference. Create the actions manually as described above.
Dataverse column not found	Use the connector's actual logical property names consistently. The template uses the names supplied in chat.
No token row / two token rows	Check the selected environment/table and the single-current-row publisher contract. Do not hide duplicates with row count 1.
Expiry check fails	Check value format and UTC/offset, publisher freshness, and the proposed margin. Do not invent a lifetime.
Axon 401	Re-read once; require a usable replacement and retry the same page once.
Axon 403	Check Reports.read and agency authorization. Token renewal alone is not a permission fix.
SharePoint access denied	Check the account behind that action's connection and its folder/file permissions. Flow ownership alone does not grant SharePoint access.
Local workbook cannot find a file	Confirm DataFolder points to the extracted data folder; check SourceFile and local file access.
Saved results appear but validation says pending or failed	Check refresh errors and timestamp; cached results may remain after a failed refresh.
Local fixture checks fail after replacing the CSV or changing the date	Restore the baseline to test it, or use an isolated custom-source copy with fixture checks disabled.
Flow succeeds but CSV workbook does not update	Do not rely on Office Scripts Refresh All for CSV queries. Use the cloud processing script or manually refresh the local workbook.
Power BI shows a different run from latest	Check completed publication, the configured source path, completed model refresh, and run/date metadata.

You have completed **local testing** when the 1,000-record source refreshes, the 757-detail baseline reconciles, and all intended scenario checks have current results.

You have completed **cloud setup** when a controlled run reads the published token securely, completes both API page loops, processes a working copy, publishes only after passing its gates, and Power BI shows matching run metadata. Only then enable the intended recurring schedule.

What has and has not been verified

• The supplied mock CSV and static expected report were previously reconciled locally: 1,000 raw / 757 uncategorized records. The source/test fixtures and query code are included for inspection.

• The clean setup workbook contains the named input tables and query catalog. It has no installed queries and is not a successfully refreshed report.

• The earlier experimental embedded-query workbook is excluded from the walkthrough package. A local Mac query-authoring attempt requested an on-premises gateway; native CSV query refresh remains unverified. The

installation steps use Windows Excel menu names.

• No live Dataverse token was read. No Power Automate flow, SharePoint publication, or Power BI refresh was deployed from this guide.

Sources and verification boundary

• User-supplied Dataverse settings and confirmed single-current-row contract from this chat. No token value was supplied or added to the guide.

• The supplied local mock CSV/report/test pack and query source in this workspace.

• Repository cloud setup, implementation guide, and API integration contract, read November 2, 2026. The API contract derives from the repository's supplied vendor documentation; live API/tenant behavior remains to be checked.

• Microsoft: Dataverse List rows.

• Microsoft: manage Power Automate connections.

• Microsoft: Office Scripts refresh limitations.

This guide does not mean a live Dataverse connection, Power Automate flow, SharePoint publication, or Power BI Service refresh has been deployed or verified. Templates remain editable and require your tenant's connection bindings and destinations.